

Database Management Systems
CO5 – Assessment Tool 3: Mock Interview

Q1.

Interviewer: “Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?”

Answer:

Query processing moves a SQL statement through four broad stages. First, parsing and translation: the parser checks the query against SQL grammar, validates that the tables and columns exist in the catalog, and translates the query into an internal form such as relational algebra. Second, optimization: the DBMS generates several logically equivalent execution plans for the same query — for example, different join orders or different access paths — and estimates the cost of each in terms of disk I/O, CPU usage, and memory. Third, plan generation: the optimizer picks the plan with the lowest estimated cost and compiles it into an executable, low-level query plan made of physical operators such as index scans, nested-loop joins, or hash joins. Finally, execution: the query evaluation engine runs this plan against the database and returns the result set. The optimizer decides the best plan by relying on statistics stored in the system catalog — table cardinalities, column selectivities, index availability, and data distribution. It uses these statistics inside a cost model to estimate the number of page I/Os (and CPU cycles) each candidate plan would need, then picks the plan with the minimum estimated total cost, typically using dynamic programming over the space of join orders so that it doesn’t have to enumerate every possible plan explicitly.

Q2.

Interviewer: “What is the difference between heuristic optimization and cost-based optimization? When would you use each?”

Answer:

Heuristic optimization applies a fixed set of transformation rules to the query tree that are known to almost always reduce cost, regardless of the actual data — for instance, pushing selection and projection operations as early as possible in the plan, or eliminating redundant operators. It doesn’t look at table statistics; it just rewrites the logical plan using rules of thumb, so it is fast and cheap to apply, but it can’t always guarantee the truly optimal plan. Cost-based optimization goes a step further: it generates multiple candidate physical plans and uses real statistics — cardinalities, selectivities, index costs — to estimate and compare their actual execution cost, then selects the cheapest one. It gives a much better plan on average but is more computationally expensive, since it may need to evaluate many join orderings and access-path combinations. In practice, both are used together: heuristic rules are applied first as a cheap first pass to simplify the query and reduce the search space, and cost-based optimization is then applied on top — especially for join ordering and choosing between index scans versus full table scans — where the actual data distribution really changes which plan is fastest.

Q3.

Interviewer: “How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.”

Answer:

In a real project I would not open a new physical connection for every request — that’s expensive. Instead, I’d configure a connection pool (for example HikariCP in a Java project) with a sensible minimum and maximum pool size, obtain a connection from the pool for each unit of work, always use parameterized/prepared statements to avoid SQL injection, wrap each unit of work in an explicit transaction with commit/rollback, and release the connection back to the pool in a finally block so it is never leaked even if an exception occurs. JDBC (Java Database Connectivity) is a Java-specific API: it lets Java applications talk to a database through a JDBC driver supplied by the database vendor, and it is platform-independent because it runs on the JVM. ODBC (Open Database Connectivity) is a more general, language-independent standard defined by Microsoft that uses native C-based drivers and a driver manager, and is commonly used by non-Java applications such as those written in C++ or by tools like Excel. Historically, a JDBC-ODBC bridge driver let Java applications reuse existing ODBC drivers, but modern projects almost always use a native, vendor-supplied JDBC driver directly for better performance and reliability.

Q4.

Interviewer: “What are ACID properties? Give a real-world example where each property is critical.”

Answer:

ACID stands for Atomicity, Consistency, Isolation, and Durability, and together they guarantee that transactions are processed reliably. Atomicity means a transaction either completes fully or not at all — in a bank transfer, if money is debited from Account A it must also be credited to Account B; if the credit step fails, the debit must be rolled back too, otherwise money simply disappears. Consistency means a transaction takes the database from one valid state to another, respecting all constraints — for example, an account balance constraint that disallows negative balances must still hold true after the transaction commits. Isolation means concurrent transactions don’t interfere with each other’s intermediate results — if two customers try to book the last seat on a flight at the same time, isolation ensures only one of them succeeds and the other sees a consistent, already-updated view rather than a half-finished booking. Durability means once a transaction commits, its effects survive even a system crash — if an ATM confirms a successful withdrawal and the system crashes a second later, that withdrawal must still be reflected in the account when the system comes back up, which is what write-ahead logging guarantees.

Q5.

Interviewer: “Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?”

Answer:

A dirty read happens when one transaction reads a value that another transaction has written but not yet committed; if the writer later rolls back, the reader has used a value that never really existed. A lost update happens when two transactions read the same value, both compute a new value based on it, and both write

back — whichever write happens last silently overwrites the other, so one update is lost. An unrepeatable read happens when a transaction reads the same row twice and gets two different values because another transaction updated and committed that row in between the two reads. Two-Phase Locking solves all three by enforcing a growing phase, where a transaction can only acquire locks, followed by a shrinking phase, where it can only release them, and never mix the two. Because a transaction must hold a shared lock while reading and an exclusive lock while writing, and (in strict 2PL) hold exclusive locks until commit, other transactions are blocked from reading uncommitted writes (preventing dirty reads), from writing to a row that is exclusively locked by another in-progress transaction (preventing lost updates), and from modifying a row that is currently read-locked by an ongoing transaction (preventing unrepeatable reads).

Q6.

Interviewer: “What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.”

Answer:

A deadlock occurs when two or more transactions are each waiting for a lock held by another transaction in the same group, so none of them can ever proceed — for instance, Transaction A holds a lock on row X and waits for row Y, while Transaction B holds a lock on row Y and waits for row X. Deadlocks are detected using a Wait-For Graph, where each node is a transaction and a directed edge from T_i to T_j means T_i is waiting for a lock held by T_j . The DBMS periodically builds this graph from the lock table and runs a cycle-detection algorithm, typically a depth-first search that tracks nodes currently on the recursion stack; if the search revisits a node still on that stack, a cycle exists and a deadlock has been found. The database then picks a victim transaction, usually the youngest or the one that has done the least work, and aborts it to break the cycle. For prevention rather than detection, two common strategies are: (1) the wait-die / wound-wait schemes, which use transaction timestamps so that only an older transaction is allowed to wait for a younger one (wait-die) or a younger transaction is forced to abort if it would need to wait for an older one (wound-wait), guaranteeing no cycles can form; and (2) resource/lock ordering, where all transactions are required to acquire locks on data items in a fixed global order, which makes a circular wait impossible by construction.

Q7.

Interviewer: “Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?”

Answer:

Immediate Update allows a transaction’s changes to be written to disk before it commits, as long as the old value is logged first (write-ahead logging). Because uncommitted changes may already be on disk, recovery after a crash needs both an UNDO pass, to roll back the writes of transactions that never committed, and a REDO pass, to reapply the writes of committed transactions that hadn’t yet been flushed. Deferred Update, by contrast, buffers all of a transaction’s writes in memory and only applies them to disk after the transaction commits. Since nothing is written to disk before commit, recovery never needs to undo anything — only a REDO pass is required for committed transactions whose writes didn’t make it to disk before the crash. Immediate Update is preferred when transactions are long-running or update more data than comfortably fits in the buffer pool, since it lets the system flush pages early to free up memory. Deferred Update is preferred

for shorter transactions where all the updates can reasonably be held in memory until commit, because it keeps recovery simpler (no undo logic needed) at the cost of holding more uncommitted data in the buffer.

Q8.

Interviewer: “Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?”

Answer:

Shadow Paging avoids using a log at all. The database keeps two page tables: a shadow page table, which is never modified while a transaction runs and always points to the last consistent, committed version of the database, and a current (working) page table, which the transaction uses. Whenever the transaction writes to a page, the DBMS never overwrites that page in place — instead it allocates a brand-new disk block, writes the updated data there, and updates only the current page table to point to this new block. To commit, the DBMS simply makes the current page table become the new shadow page table through a single atomic pointer swap; to abort, it just discards the current page table and the newly allocated blocks, and the original shadow table is still intact and consistent. The main advantage is that recovery becomes almost instantaneous and very simple, since there is no log to replay — after a crash, the system just falls back to whichever page table was last confirmed as “current,” and there is no separate undo/redo processing needed. The limitations are that it causes disk fragmentation, since old pages are scattered and must be garbage-collected once no longer referenced; it makes supporting concurrent transactions harder, because sharing a single working page table across multiple simultaneous transactions is awkward; and every commit requires flushing potentially many changed pages before the pointer swap, which can be more expensive than simply appending to a sequential log, as log-based schemes like ARIES do.

Q9.

Interviewer: “What is a serializability schedule? How do you test for conflict serializability using a precedence graph?”

Answer:

A serializable schedule is a concurrent (interleaved) execution of multiple transactions whose final effect on the database is exactly the same as if those transactions had been executed one after another in some serial order, even though they actually ran concurrently. Serializability is the correctness criterion the database tries to guarantee, because a serial execution is trivially correct and we want concurrency to never introduce anomalies that a serial run wouldn't have. To test for conflict serializability specifically, we build a precedence graph: each transaction becomes a node, and we draw a directed edge from T_i to T_j whenever T_i has an operation that conflicts with a later operation of T_j on the same data item — conflicting meaning at least one of the two operations is a write, and they access the same item. After adding an edge for every such conflicting pair in the schedule, we check whether the resulting graph contains a cycle. If the graph is acyclic, the schedule is conflict-serializable, and a valid equivalent serial order can be obtained by performing a topological sort of the graph. If the graph contains a cycle, the schedule is not conflict-serializable, because the cycle represents a contradictory ordering requirement that no serial schedule could satisfy.

Q10.

Interviewer: “Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?”

Answer:

All three variants share the core 2PL rule: a transaction has a growing phase in which it only acquires locks, followed by a shrinking phase in which it only releases them, and it can never acquire a new lock once it has released any lock. They differ in exactly when locks are released. Basic 2PL releases locks as soon as a transaction no longer needs them, which can be well before commit; this maximizes concurrency but allows cascading rollbacks, because another transaction may have already read a value that gets rolled back later. Strict 2PL holds all exclusive (write) locks until the transaction commits or aborts, while shared (read) locks may still be released earlier; this prevents other transactions from reading uncommitted writes, eliminating cascading rollbacks caused by dirty reads, though a transaction could still read a value that is later changed by another. Rigorous 2PL is the strictest: it holds both shared and exclusive locks until commit or abort, guaranteeing that transactions appear to execute in their commit order, which makes recovery simpler and schedules strict as well as serializable. Rigorous 2PL is the variant most commonly implemented in commercial database systems, because holding all locks until the end of the transaction is straightforward to implement correctly, avoids cascading rollbacks entirely, and makes the interaction between concurrency control and the recovery manager much simpler to reason about — the small loss in concurrency compared to Basic or Strict 2PL is considered a worthwhile trade-off for that simplicity and safety.

Interview Score

Criteria	Weightage	Level Achieved	Score
Technical Knowledge	25	Level 4	23
Problem Solving	20	Level 4	18
Analytical Thinking	15	Level 4	14
Communication	15	Level 4	14
Confidence	15	Level 3	13
Response to Questions	10	Level 4	10
Total	100		92

Overall Score: 92 / 100